🧠 DSA Notes

Mathematics Basics — Beginner Friendly

Only the maths you actually need for DSA & problem solving

📌 **Important Rule — Read This First**

You do NOT need deep mathematics for DSA.

Focus on: pattern recognition • logic • formula usage • complexity understanding

Ignore: deep derivations • proofs • advanced algebra

1️⃣ 📈 Arithmetic Progression (AP)

📖 Definition

A sequence where the difference between consecutive numbers stays the same.

2	4	6	8	10
	+2	+2	+2	+2

📌 Formula

```
nth term  =  a + (n - 1) × d
a  =  first term
d  =  common difference  (how much you add each time)
n  =  position of the term you want
```

✅ Worked Example — Find the 5th term of: 2, 4, 6, 8, 10

```
a = 2   (first term)
d = 2   (difference: 4-2 = 2)
n = 5   (we want the 5th term)
────────────────────────────
= a + (n - 1) × d
= 2 + (5 - 1) × 2
```

```
= 2 + 4 × 2
= 2 + 8  =  10  ✅
```

🎯 **Easy Trick:** AP = keep ADDING the same number each step.

2️⃣ 📈 Geometric Progression (GP)

📖 Definition

A sequence where you multiply by the same number each step.

2	4	8	16	32
	×2	×2	×2	×2

📌 Formula

```
nth term  =  a × r^(n - 1)
a  =  first term
r  =  common ratio  (how much you multiply each time)
n  =  position of the term you want
```

✅ Worked Example — Find the 4th term of: 2, 4, 8, 16

```
a = 2   (first term)
r = 2   (ratio: 4÷2 = 2)
n = 4   (we want the 4th term)
────────────────────────────
= a × r^(n - 1)
= 2 × 2^(4 - 1)
= 2 × 2³
= 2 × 8  =  16  ✅
```

🎯 **Easy Trick:** GP = keep MULTIPLYING the same number each step.

📌 **AP vs GP: AP adds a constant. GP multiplies a constant. Both show up constantly in loop analysis!**

3️⃣ 🔢 Mean (Average)

📖 Definition

The mean is the average — the fair share value if you spread everything equally.

```
Mean  =  (sum of all values)  ÷  (count of values)
```

✅ Worked Example

```
Values: 1, 2, 3
Sum    = 1 + 2 + 3 = 6
Count  = 3
────────────────────
Mean   = 6 ÷ 3  =  2  ✅
```

4️⃣ 🔢 Median (Middle Value)

📖 Definition

The median is the middle value after sorting the data in order.

✅ Worked Example

```
Values (unsorted): 7, 1, 3
Step 1 → Sort:     1, 3, 7
Step 2 → Find middle value
────────────────────────
Median = 3  ✅
```

Mean	Median
Adds all values and divides by count	Picks the middle value after sorting
Affected by very large/small values (outliers)	More stable — outliers don't shift it
Example: 1,2,100 → mean = 34.3	Example: 1,2,100 → median = 2

5️⃣ 🔢 Prime Numbers

📖 Definition

A prime number is divisible ONLY by 1 and itself — nothing else.

```
✅ Primes:  2, 3, 5, 7, 11, 13, 17 ...

❌ Not prime:  4  (divisible by 1, 2, 4)
❌ Not prime:  9  (divisible by 1, 3, 9)
```

⚡ Prime Optimization Trick

Check factors only up to √n. Why? Because factors always come in pairs — if p is a factor, n/p is also a factor. After √n, pairs repeat.

```
Is 37 prime?
√37 ≈ 6.08 → check up to 6
Check: 2, 3, 4, 5, 6
None divide 37 cleanly.
───────────────────────
37 is PRIME ✅
```

6️⃣ 🔢 Factors

📖 Definition

Factors of a number are all the integers that divide it completely (no remainder).

Factors of 12:
1 2 3 4 6 12

Why? Because: 12÷1=12 | 12÷2=6 | 12÷3=4 | 12÷4=3 | 12÷6=2 | 12÷12=1

> 💡 **DSA use: Finding factors efficiently (loop up to √n) is a common pattern in number theory problems.**

7️⃣ 🔢 HCF / GCD (Highest Common Factor)

📖 Definition

HCF (also called GCD) is the largest number that divides two numbers without remainder.

✅ Worked Example — HCF of 12 and 16

Factors of 12	Factors of 16
`1, 2, 3, 4, 6, 12`	`1, 2, 4, 8, 16`
Common factors: `1, 2, 4` **HCF = 4**	

> 💡 **In code: Euclidean Algorithm — gcd(a, b) = gcd(b, a % b). Keep going until b = 0. Much faster than listing factors!**

8️⃣ 🔢 LCM (Least Common Multiple)

📖 Definition

LCM is the smallest number that is a multiple of both given numbers.

✅ Worked Example — LCM of 4 and 6

```
Multiples of 4:   4,  8,  12,  16,  20 ...
Multiples of 6:   6,  12, 18,  24  ...
                      ↑
Smallest common → 12
LCM(4, 6)  =  12  ✅
```

```
Important Relation:  LCM × HCF  =  a × b

Example: LCM(4,6) × HCF(4,6)  =  4 × 6
```

```
        12          ×  2          =  24  ✅
```

9️⃣ 🔢 Factorial

📖 Definition

Factorial means: multiply all whole numbers from n down to 1.

```
n!  =  n × (n-1) × (n-2) × ... × 2 × 1
```

✅ Examples

```
5! = 5 × 4 × 3 × 2 × 1 = 120
4! = 4 × 3 × 2 × 1 = 24
3! = 3 × 2 × 1 = 6
────────────────────────────
Special cases:  0! = 1   and   1! = 1
```

> 💡 **DSA use: Factorials appear in Permutation and Combination formulas. Also: n! grows EXTREMELY fast — 20! is already 2.4 quintillion.**

🔟 🔀 Permutation

📖 Definition

Permutation = arrangement where ORDER MATTERS. Swapping positions gives a different arrangement.

```
Choose Captain and Vice-Captain from: A, B

Option 1:  Captain = A,  Vice-Captain = B
Option 2:  Captain = B,  Vice-Captain = A

→ These are TWO different arrangements.
→ Order changed = new result.
```

> 📌 **Real world: PIN codes, passwords, race positions — order matters. 1234 ≠ 4321.**

1️⃣1️⃣ 🎲 Combination

📖 Definition

Combination = selection where ORDER DOES NOT MATTER. Swapping gives the same group.

```
Choose 2 team members from: A, B

Option 1:  { A, B }
Option 2:  { B, A }

→ Same group. Only ONE way to choose.
→ Order doesn't change the team.
```

Permutation	Combination
Order MATTERS	Order does NOT matter
A,B ≠ B,A	A,B = B,A
PIN codes, ranking, seating	Teams, committees, choosing items

1️⃣2️⃣ 🔢 Modular Arithmetic

📖 Definition

Modulo (%) gives you the REMAINDER after division. Think of it as: what's left over?

> 💡 **Real-world analogy: A clock shows 12 hours. After 12, it resets to 1. That's modulo — wrapping around!**

✅ Examples — Step by Step

Expression	Closest multiple	Subtract	Result
`7 % 2`	3 × 2 = 6	7 - 6 = 1	**1**
`17 % 3`	5 × 3 = 15	17 - 15 = 2	**2**

`10 % 5`	2 × 5 = 10	10 - 10 = 0	0

```
7  % 2  = 1   ← 7 = 3×2 + 1  (remainder 1)
17 % 3  = 2   ← 17 = 5×3 + 2 (remainder 2)
10 % 5  = 0   ← 10 = 2×5 + 0 (remainder 0, divides evenly)
```

> 📌 **DSA use: Modulo is everywhere — checking even/odd (n%2), cycling through indices, hash functions, preventing integer overflow in large computations.**

📊 Quick Reference — All Formulas at a Glance

Concept	Formula / Key Fact
AP nth term	`a + (n-1)d`
GP nth term	`a × r^(n-1)`
Mean	`sum / count`
Factorial	`n × (n-1) × … × 1`
LCM × HCF	`= a × b`
Prime check	`factors up to √n`
Modulo (%)	`remainder`